

Neural Networks and Deep Learning

W. Evan Johnson, Ph.D.

Professor, Division of Infectious Disease

Director, Center for Data Science

Director, Center for Biomedical Informatics and Health AI

Rutgers University – New Jersey Medical School

2026-06-18

“Biology / Biomedicine is a Data Science.”

— W. Evan Johnson

Addendum: don't avoid the math!

“With great power comes great responsibility.”

— Uncle Ben, *Spider-Man*

A Quick Map of “AI”

Where do neural networks fit in the broader AI landscape?

- ▶ **Narrow AI** (all AI today): does one specific task well — Face ID, spam filters, recommendations, Google Maps, ChatGPT
- ▶ **Generative AI**: creates new content from learned patterns — text, images, music, video, voice
- ▶ **Agentic AI** (emerging): takes actions, makes plans, and uses tools to accomplish goals — schedules meetings, sends email, runs tasks automatically

A Quick Map of “AI”

- ▶ **Artificial General Intelligence (AGI)**: human-level reasoning across many tasks — **theoretical; does not exist** (movies exaggerate it)
- ▶ **Artificial Super Intelligence (ASI)**: hypothetical intelligence that would *surpass* humans across virtually all domains — **also does not exist**

Everything we build today — including the neural networks in this lecture — is **Narrow AI**.

Real (Human) Intelligence

The ability to think, choose, create, understand meaning, and act with moral responsibility:

- ▶ **Agency** — we choose
- ▶ **Moral judgment** — right vs. wrong
- ▶ **Creativity** — truly new ideas
- ▶ **Understanding meaning**
- ▶ **Empathy and relationships**
- ▶ **Spiritual capacity**

“AI will not replace us, it will amplify us.”

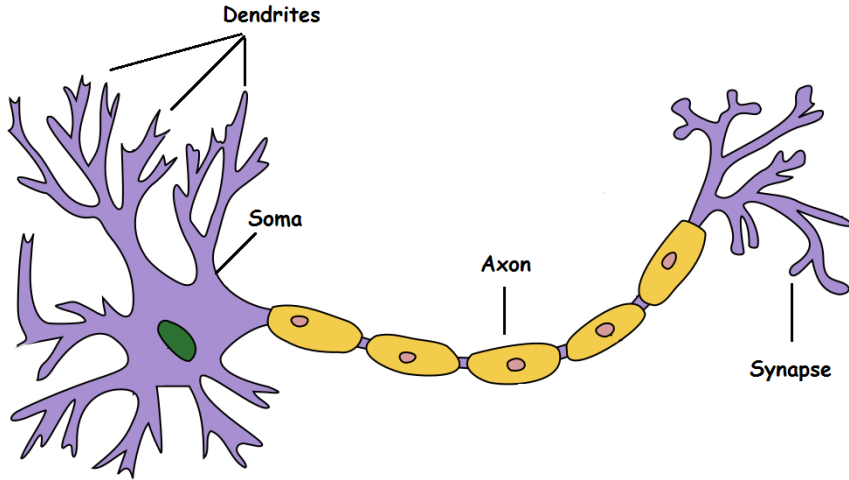
— Pedro Domingos

Introduction to Neural Networks

The brain has billions of neurons, each a simple processor, wired into a **biological neural network** that passes electrical signals from neuron to neuron:

- ▶ **Dendrites** — receive input signals
- ▶ **Soma** (cell body) — sums the inputs
- ▶ **Axon** → **synapses** — pass the output on to other neurons

Introduction to Neural Networks



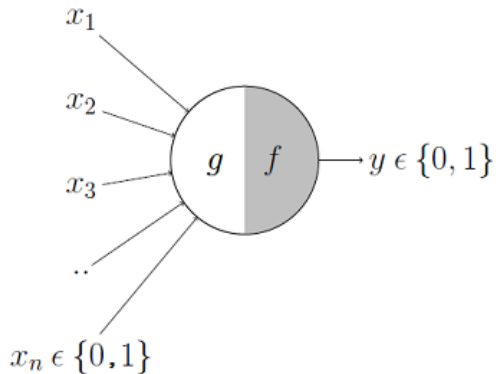
Introduction to Neural Networks

McCulloch & Pitts (1943–44) — the first mathematical model of a neuron:

- ▶ Takes inputs, processes them, and returns an output (thresholds + weights)
- ▶ **No layers and no training rule**
- ▶ Showed a neural net could *in principle* compute any function a computer could — framing the brain as a computing device

Introduction to Neural Networks

The McCulloch-Pitts neuron model looked something like this:



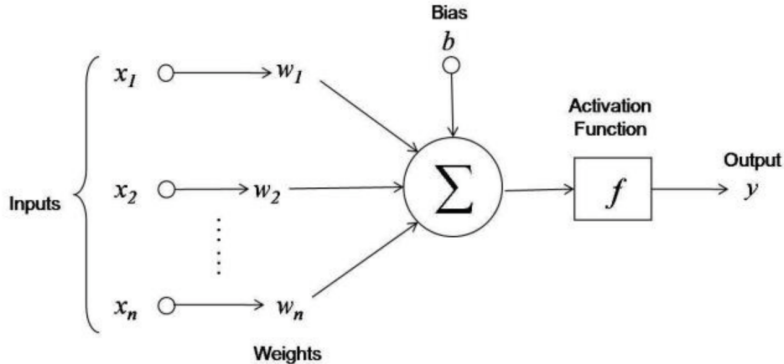
Introduction to Neural Networks

The **Perceptron** — the first *trainable* neural network:

- ▶ **1957 (Rosenblatt)**: one layer of adjustable weights and thresholds, between input and output layers
- ▶ **1959 (Minsky & Papert)**: showed common computations were *impractically slow* on perceptrons — research stalls

Introduction to Neural Networks

The Perceptron model looked something like this:



Introduction to Neural Networks

The pieces of a neuron:

- ▶ **Inputs** $x_1, \dots, x_N$ — the predictors
- ▶ **Weights** $w_1, \dots, w_N$ **and bias** b — adjustable parameters, summed into the net input
- ▶ **Activation function** — maps that sum to the output

Learning = adjusting the weights and bias from the data.

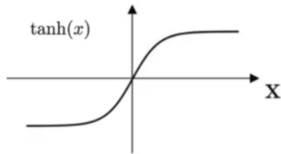
Introduction to Neural Networks

The **activation function** maps a neuron's weighted input to its output — giving the network its **non-linearity** (often just one line of code):

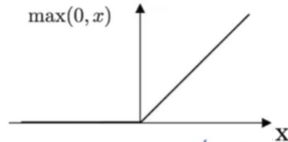
- ▶ **Identity** — $\text{output} = \text{input}$ (linear)
- ▶ **Binary step** — on/off above a threshold (classifier)
- ▶ **Sigmoid / tanh** — smooth S-curve for probabilities
- ▶ **ReLU** — $\max(0, x)$; the modern default

Introduction to Neural Networks

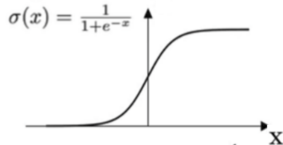
Hyper Tangent Function



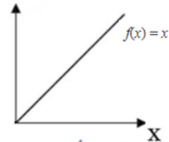
ReLU Function



Sigmoid Function

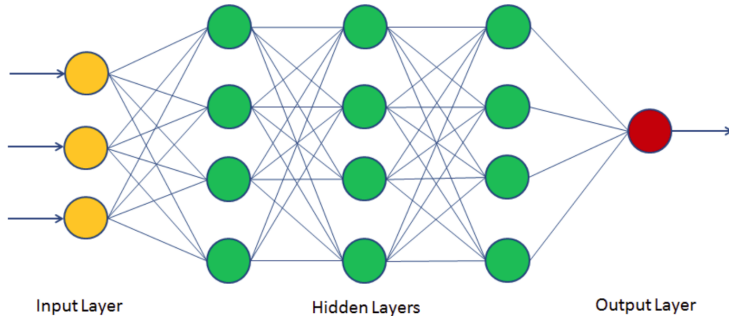


Identity Function



Introduction to Neural Networks

Modern **neural networks** are algorithms comprising multiple layers of connected input/output units (neurons) in which each connection has a weight associated with it.



How Does a Neural Network Learn?

Training a network means finding the **weights** that make its predictions match the data. Three ingredients:

1. A **loss** $\ell(y, \hat{y})$: how wrong is the prediction?
2. **Backpropagation**: how does each weight affect the loss?
3. **Gradient descent**: nudge each weight to reduce the loss.

“AI is not magic; it’s math.”

— Peter Norvig

The Chain Rule

The Famous CHAIN RULE

$$\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$$



The Chain Rule

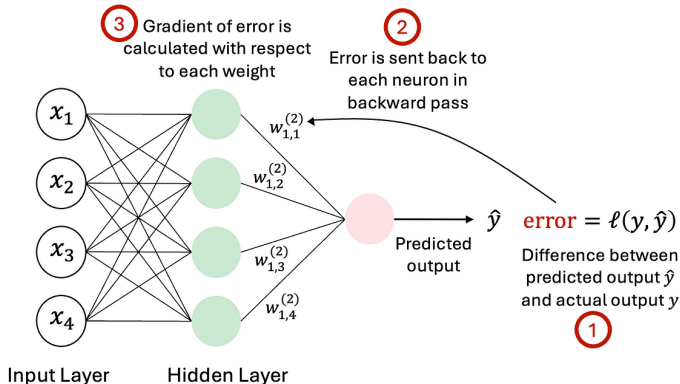
A neural network is just **functions composed inside functions**.
The derivative of a composition multiplies the derivatives of each layer:

$$f' \circ g \circ h \circ k(x) \times g' \circ h \circ k(x) \times h' \circ k(x) \times k'(x)$$



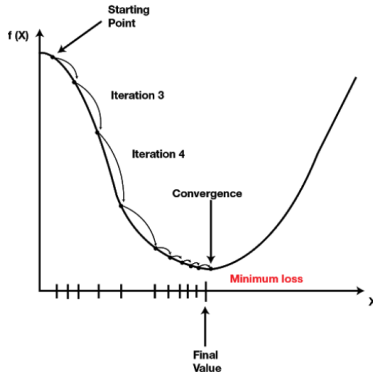
Backpropagation

The chain rule lets us send the error **backward** through the network and compute the gradient of the loss with respect to **every weight**:



Gradient Descent

Each step adjusts the weights in the direction that **decreases the loss**, iterating until it converges to a minimum:



Multilayer Perceptron Neural Network in R

Suppose we have students' technical knowledge (TKS), communication skill score (CSS), and placement status (Placed):

```
TKS=c(20,10,30,20,80,30)
CSS=c(90,20,40,50,50,80)
Placed=c(1,0,0,0,1,1)
df=data.frame(TKS,CSS,Placed)
```

Multilayer Perceptron Neural Network in R

```
knitr::kable(df)
```

TKS	CSS	Placed
20	90	1
10	20	0
30	40	0
20	50	0
80	50	1
30	80	1

Multilayer Perceptron Neural Network in R

Fit the multilayer perceptron neural network:

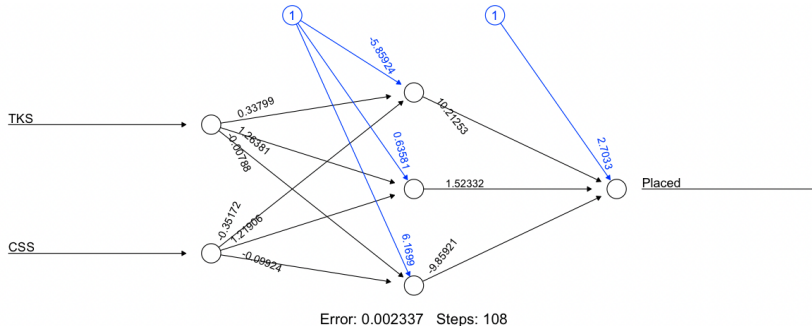
```
suppressPackageStartupMessages(library(neuralnet))
set.seed(0)
nn=neuralnet::neuralnet(Placed~TKS+CSS,data=df, hidden=3,
                        act.fct = "logistic", linear.output = FALSE)
names(nn)
```

## [1]	"call"	"response"	"covariate"
## [4]	"model.list"	"err.fct"	"act.fct"
## [7]	"linear.output"	"data"	"exclude"
## [10]	"net.result"	"weights"	"generalized.weights"
## [13]	"startweights"	"result.matrix"	

Multilayer Perceptron Neural Network in R

We can plot or neural network:

```
plot(nn)
```



Multilayer Perceptron Neural Network in R

Creating a test set:

```
TKS=c(30,40,85)  
CSS=c(85,50,40)  
test=data.frame(TKS,CSS)
```

Multilayer Perceptron Neural Network in R

```
knitr::kable(test)
```

TKS	CSS
30	85
40	50
85	40

Multilayer Perceptron Neural Network in R

Prediction using neural network:

```
Predict=compute(nn,test)
Predict$net.result
```

```
##           [,1]
## [1,] 0.4013943
## [2,] 0.4014894
## [3,] 0.4974918
```

Multilayer Perceptron Neural Network in R

Converting probabilities into binary classes setting threshold level 0.5:

```
prob <- Predict$net.result  
pred <- ifelse(prob>0.5, 1, 0)  
pred
```

```
##      [,1]  
## [1,]    0  
## [2,]    0  
## [3,]    0
```

Multilayer Perceptron Neural Network in R

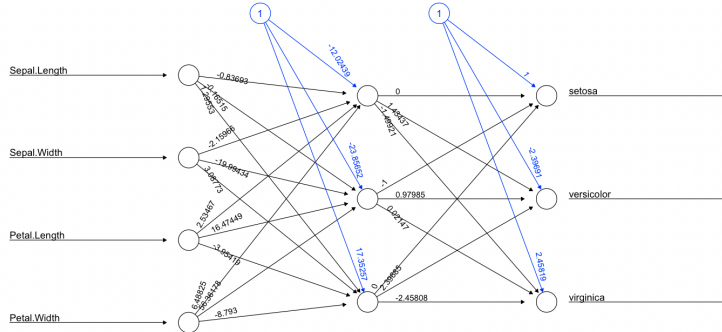
Now, using the iris dataset:

```
set.seed(0)
nn_iris <- neuralnet(Species ~ ., data=iris, hidden=3)
```

Multilayer Perceptron Neural Network in R

Plotting the Neural Network:

```
plot(nn_iris)
```



Multilayer Perceptron Neural Network in R

Now, doing the same thing using 'caret':

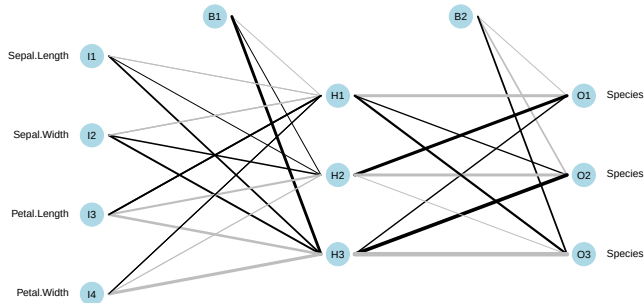
```
set.seed(0)
nn_caret <- caret::train(Species~., data = iris,
                        method = "nnet", linout = TRUE,
                        trace = FALSE)
ps <- predict(nn_caret, iris)
confusionMatrix(ps, iris$Species)$overall["Accuracy"]
```

```
## Accuracy
## 0.9733333
```

Multilayer Perceptron Neural Network in R

Plotting the 'caret' neural network:

```
NeuralNetTools::plotnet(nn_caret)
```



Introduction to Neural Networks

Modern neural networks **learn by example** from training data — adjusting weights across a network of artificial neurons to capture **non-linear** patterns.

- ▶ Best for tasks easy for humans but hard to program: **pattern recognition**
- ▶ Applications: image classification (cats vs. dogs), OCR, object detection

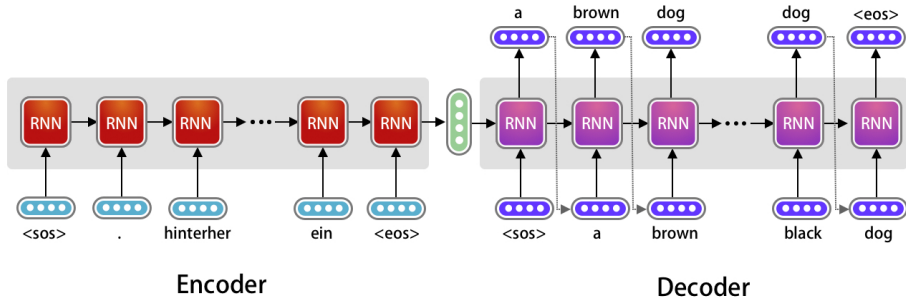
Introduction to Neural Networks

Two common architectures:

- ▶ **Feedforward** — non-recursive; signals travel one way, layer to layer, toward the output
- ▶ **Feedback (recurrent)** — contains loops; signals travel in both directions

Recurrent Neural Networks (RNNs)

Feedback neural networks contain cycles. Signals travel in both directions by introducing loops in the network. Feedback neural networks are also known as *recurrent neural networks*.



Introduction to Neural Networks

There are many other formulations of **deep learning** neural networks:

- ▶ Perceptron/Multilayer Perceptron
- ▶ Feedforward Neural Network
- ▶ Recurrent Neural Network
- ▶ Convolutional Neural Network
- ▶ Radial Basis Functional Neural Network
- ▶ LSTM – Long Short-Term Memory
- ▶ Sequence to Sequence Models
- ▶ Modular Neural Network

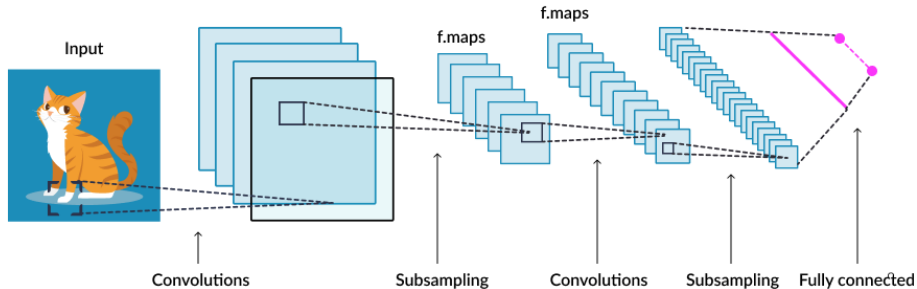
Convolutional Neural Networks

A **CNN** (ConvNet) is a neural network specialized for **grid-like data** such as images:

- ▶ **Shared-weight filters (kernels)** slide across the input → **feature maps**
- ▶ **Translation-invariant**: finds a pattern anywhere in the image
- ▶ Builds complexity hierarchically — a **regularized** form of an MLP

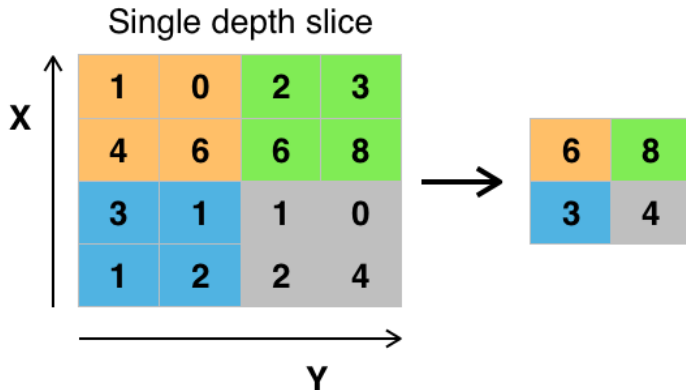
Convolutional Neural Networks

- ▶ **Filters are learned automatically** — no hand-engineering
- ▶ Applications: image & video recognition, classification & segmentation, medical imaging, NLP, time series



Convolutional Neural Networks

Maximum filter example (or **pooling**):



Convolutional Neural Networks

Sliding (overlapping) filter example (**kernel**, **stride** and **padding**):

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

1	1	1	0	0
0	1	1 _{x1}	1 _{x0}	0 _{x1}
0	0	1 _{x0}	1 _{x1}	1 _{x0}
0	0	1 _{x1}	1 _{x0}	0 _{x1}
0	1	1	0	0

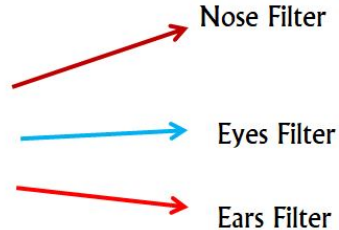
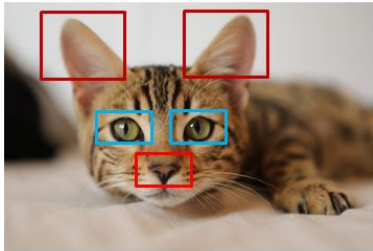
Image

4	3	4
2	4	3

Convolved
Feature

Convolutional Neural Networks

Now applying this to cats:



Convolutional Neural Networks

More realistically:



Cat



Cat



Representation

Convolutional Neural Networks

And as a classifier:



Cat



Not a cat

Recurrent Neural Networks

- ▶ A type of neural network for **sequential data**
- ▶ Maintains **memory of previous inputs** using hidden states
- ▶ Commonly used for:
 - ▶ Natural Language Processing (NLP)
 - ▶ Time Series Forecasting
 - ▶ Speech Recognition

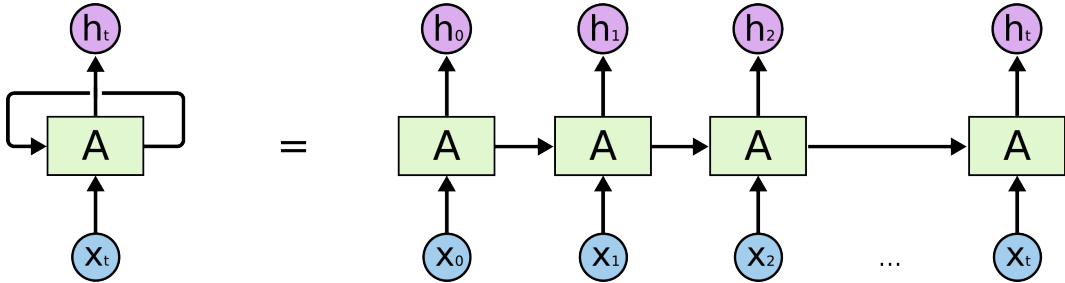
Why Not Feedforward Networks?

- ▶ Standard neural networks assume **independent inputs**
- ▶ But sequences (like text) have **temporal dependencies**
- ▶ Example:
 - ▶ Sentence: *"The cat sat on the ____."*
 - ▶ You need previous words to predict the blank

RNN Architecture

- ▶ Loops over time: same weights applied at each time step
- ▶ Input at time t : x_t
- ▶ Hidden state: $h_t = f(Wx_t + Uh_{t-1} + b)$
- ▶ Output: $y_t = g(Vh_t + c)$

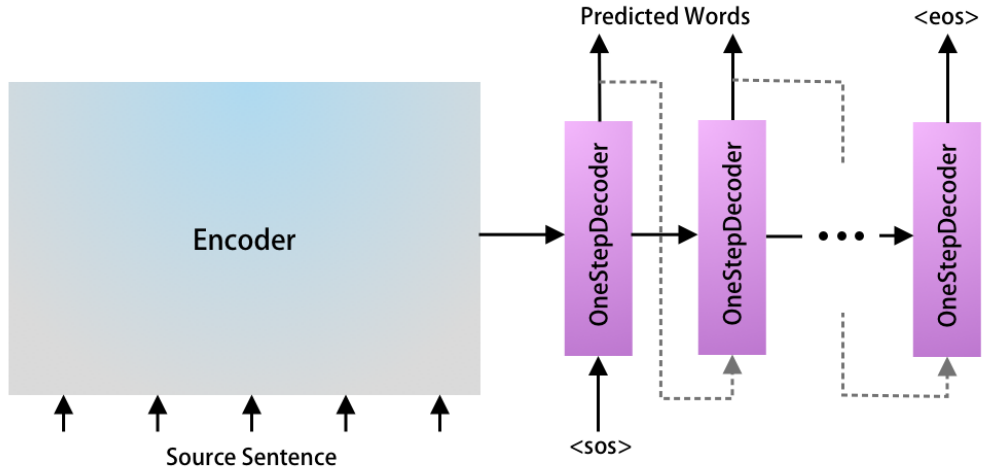
Diagram of RNN (Unrolled)



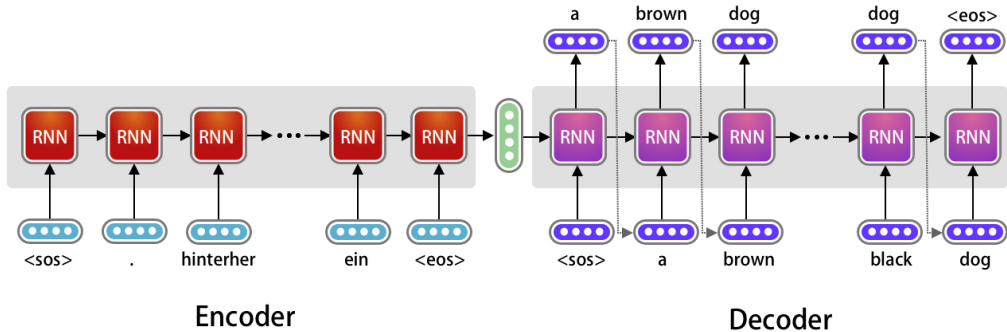
Challenges of RNNs

- ▶ Vanishing/exploding gradients during training
- ▶ Hard to learn long-term dependencies
- ▶ Solutions:
 - ▶ LSTM (Long Short-Term Memory)
 - ▶ GRU (Gated Recurrent Unit)

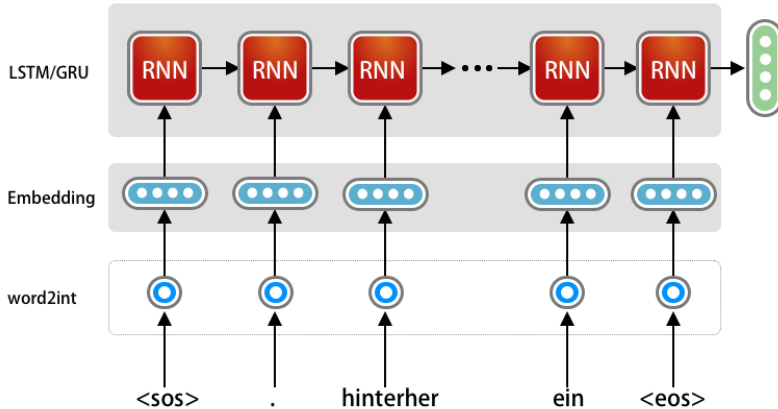
RNN for Language Translation



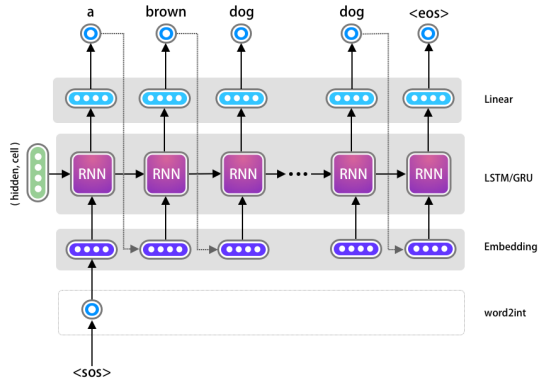
RNN for Language Translation



RNN for Language Translation



RNN for Language Translation



From RNNs to Transformers

- ▶ RNNs process inputs **sequentially** – limits parallelism
- ▶ Transformers process the **entire sequence at once**
- ▶ Introduced in “**Attention is All You Need**” (Vaswani et al., 2017)

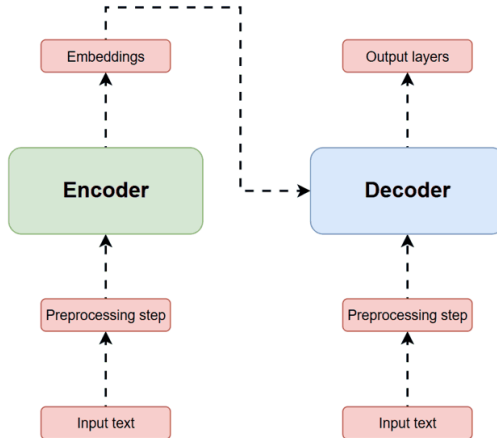
Transformers: A Very Advanced Autocomplete

GPT = **G**enerative **P**retrained **T**ransformer. At its core, it just predicts the next word:

- ▶ You type a prompt
- ▶ The model looks at **all** the words at once
- ▶ It assigns **probabilities** to possible next words
- ▶ It picks a likely next word — then repeats, again and again

ChatGPT is, in essence, a *very advanced autocomplete*.

Transformers: Encoder–Decoder



Key Idea: Attention

- ▶ Attention allows the model to **focus on relevant parts** of the input
- ▶ Computes attention scores between all pairs of tokens
- ▶ Example: In translation, aligns source and target words

Transformer Architecture

- ▶ Consists of **Encoder** and **Decoder** stacks
- ▶ Each block has:
 - ▶ Multi-head Self-Attention
 - ▶ Feedforward Neural Network
 - ▶ Residual connections & Layer normalization

Self-Attention (Scaled Dot Product)

- ▶ For queries Q , keys K , values V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- ▶ Learns how each token relates to others in the sequence

Multi-Head Attention

- ▶ Multiple attention “heads” allow the model to learn different relationships
- ▶ Outputs from each head are concatenated and linearly transformed

Positional Encoding

- ▶ Transformers have no recurrence or convolution
- ▶ Positional encoding injects **order information** into the input embeddings
- ▶ Common method: use sine/cosine functions of different frequencies

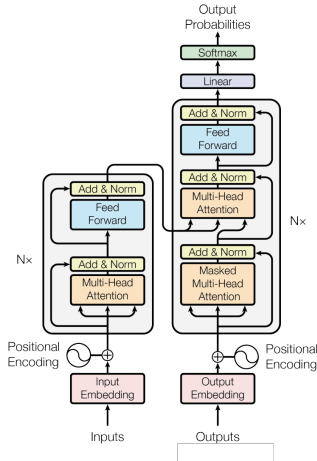
Advantages of Transformers

- ▶ **Highly parallelizable** – faster training
- ▶ Better at learning **long-range dependencies**
- ▶ Backbone of modern NLP: BERT, GPT, T5, etc.

Transformer Architecture

BERT

Encoder



GPT

Decoder

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.1
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
##
## locale:
## [1] en_US/en_US/en_US/C/en_US/en_US
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] neuralnet_1.44.2 lubridate_1.9.5 forcats_1.0.1 stringr_1.6.0
## [5] dplyr_1.2.1      purrr_1.2.2      readr_2.2.0     tidyr_1.3.2
## [9] tibble_3.3.1     tidyverse_2.0.0  caret_7.0-1     lattice_0.22-9
## [13] ggplot2_4.0.3    knitr_1.51
##
## loaded via a namespace (and not attached):
## [1] rstatix_0.2.6      rfun_0.5.2         rmarkdown_1.3.2
```